HANDOFF TÉCNICO ESTADO VERIFICADO TEXTURAS BC7 NO VULKAN

ASTRO BOT — PORT PARA PC

Relatório de progresso, decisões técnicas e ponto exato de retomada do projeto de recompilação/emulação de ASTRO BOT (PS5) para Windows.

Data: 23 de outubro de 2026

Plataforma alvo: Windows x64 / Vulkan

Jogo base: PPSA21567 (PS5)

Compilador: LLVM-MinGW / Clang++ UCRT

Executável: build\clang\AstroBot_PC.exe

Build atual: concluído; textura BC7 12 mips + captura de frame

1. Sumário executivo

O projeto já possui uma base funcional de janela, interface GDI, viewport Vulkan real, swapchain, pipeline gráfico, buffers de malha e apresentação de frames. A renderização do viewport foi validada por logs e está estável.

Na frente de assets, a **decodificação real do contêiner GNF** está integrada ao pipeline: o carregador lê o cabeçalho v4/v5, extrai dimensões, mips, tile mode e formato, e realiza o **detile padrão de 64 KB** de todos os mips das texturas BC7/BC4. Nesta etapa, a textura passou a ser **enviada para a GPU** (imagem BC7, staging buffer, sampler e descriptor set) e o shader já recebe UVs e amostra o `sampler2D`.

> **Ponto de retomada:** a textura difusa BC7 do corpo (`astro_lowpoly_costume_d.gnfp`) com 12 mips é amostrada com UVs validados em `[0,1]` e confirmada por captura de frame. Os 8 ranges de índice e os 4 materiais JXMD já são extraídos e desenhados separadamente; o foco agora é mapear material→textura (corpo/olhos), escolher sampler por material e integrar os demais mapas (normal, roughness, AO).

2. Estado atual do projeto

Área	Status	Evidência / observação
Build Windows/Clang	**Concluído**	`build_clang.bat` finaliza com “Build Clang CONCLUIDA com SUCESSO”.
Janela e interface GDI	**Funcional**	Janela principal, painel lateral, FPS e catálogo de recursos.
Viewport Vulkan	**Funcional**	HWND filho `AstroVulkanViewport`, swapchain 714x438, 3 imagens, formato 44 (BGRA8 UNORM).
Pipeline gráfico	**Funcional**	`vkCreateGraphicsPipelines` OK; primeiro frame apresentado.
Malha no viewport	**Funcional**	Buffers com 1236 vértices e 5598 índices; desenho indexado ativo.
Textura GNF — metadados	**Implementado**	Parser v4/v5 real em `asset_loader.cpp`.
Textura GNF — detile	**Implementado**	Standard 64 KB para BC7 (16 B/bloco) e BC4 (8 B/bloco); previews offline validados.
Upload de textura no Vulkan	**Validado**	`VkImage` BC7, sampler, descriptor set e barreira; captura de frame confirma a amostragem.
Compressão BC no dispositivo	**Habilitado**	`textureCompressionBC` consultado e ativado em `vkCreateDevice`.
Shader com amostragem	**Implementado**	UV no location 2; fragment shader amostra `sampler2D` (SPIR-V regerado).
Cadeia de mips GNF	**Implementado**	12/12 mips carregados e enviados; 5.592.432 bytes no total.
Captura de frame Vulkan	**Implementado**	`ASTRO_CAPTURE_FRAME=1` gera `astro_frame_capture.bmp` via `vkCmdCopyImageToBuffer`.
Malha JXMD definitiva	**Parcial**	Há parser e fallback procedural; geometria real ainda precisa ser consolidada.

3. Arquitetura e fluxo atual

```
main.cpp
  ├─ janela Win32 + GDI
  ├─ cria HWND filho "AstroVulkanViewport"
  └─ VulkanBackend::Initialize(hwnd, largura, altura, vsync)
        ├─ instância / physical device / device / fila
        ├─ surface + swapchain + image views
        ├─ depth buffer + render pass + framebuffers
        ├─ pipeline (astro.vert + astro.frag embutidos em SPIR-V)
        └─ buffers de vértice/índice

AssetLoader
  ├─ Initialize(dataRoot)
  ├─ LoadJxmd(...)  → malha
  └─ LoadGnf(...)   → metadados + mip 0 destilado em GnfTexture.data
```

O viewport é redesenhado em `BeginFrame()` / `EndFrame()`. A câmera e o modelo são controlados por `SetModelTransform(rotY, rotX, zoom, jump)`. A transformação usa matrizes próprias (`Mat4`, `Perspective`, `RotateX/Y`) e push constants.

4. O que já foi implementado

4.1. Base de plataforma e interface

- Janela principal Win32 com interface GDI, painel lateral e medidor de FPS.
- HWND filho dedicado ao Vulkan, com classe `AstroVulkanViewport`.
- Cálculo de retângulo do viewport: `left=31`, `top=172`, `vpW=(largura-70)*3/5`, `bottom=altura-71`.
- Redimensionamento e recriação de swapchain em `VK_ERROR_OUT_OF_DATE_KHR`.

4.2. Backend Vulkan

- Criação de instância, surface Win32, escolha de GPU e fila gráfica.
- Swapchain com 3 imagens e formato BGRA8 UNORM (44).
- Image views, depth buffer, render pass e framebuffers.
- Pipeline gráfico com shaders SPIR-V embutidos em `shader_bytecode.h`.
- Sincronização com semáforos e fence; apresentação via `vkQueuePresentKHR`.
- Buffers de vértice/índice e desenho indexado.
- Infraestrutura de textura: `VkImage`, `VkImageView`, `VkSampler`, staging buffer, descriptor set layout/pool/set e barreiras.
- Captura determinística de frame via `vkCmdCopyImageToBuffer` (variável `ASTRO_CAPTURE_FRAME`), contornando `PrintWindow`.

4.3. Correções críticas realizadas

Problema	Causa	Correção aplicada
`vkCreateImage` retornava -13	`VkImageCreateInfo.extent` declarado como `VkExtent2D`	Alterado para `VkExtent3D` (struct correto).
Access violation 0xC0000005 em `vkCreateGraphicsPipelines`	`pSpecializationInfo` ausente na struct de estágio	Campo adicionado e structs zeradas corretamente.
Shader inválido / leitura excessiva	`codeSize` multiplicado por `sizeof(uint32_t)`	Usado o tamanho em bytes de `kAstrovertSpirvSize` / `kAstrofragSpirvSize`.
Tipos Vulkan incompletos	Faltavam flags e handles	Adicionados `VkMemoryPropertyFlags`, `VkDescriptorSetLayout`, `VK_WHOLE_SIZE`.
Formato de profundidade	Formato anterior inadequado	`VK_FORMAT_D24_UNORM_S8_UINT`.
Textura BC7 invisível / inválida	`textureCompressionBC` não habilitado na criação do dispositivo	`VkPhysicalDeviceFeatures` completo, consulta de features e `pEnabledFeatures` em `vkCreateDevice`.
Upload copiava apenas uma linha	`VkBufferImageCopy.imageOffset` declarado como `VkOffset2D` (8 bytes) em vez de `VkOffset3D` (12 bytes), deslocando `imageExtent`	Struct corrigida para `VkOffset3D`; upload passa a copiar todos os mips e a captura lê o frame inteiro.

4.4. Carregamento de assets

- `AssetLoader::Initialize` localiza a raiz de dados, carrega a malha do Astro e cataloga vídeos/níveis.
- `LoadJxmd` tenta o parser real e usa fallback procedural quando necessário.
- `LoadGnf` foi reescrito para o formato real do jogo (detalhado na seção 5).
- Texturas do personagem carregadas no cache: `astro_lowpoly_costume_d.gnfp` (difusa do corpo) e `lowpoly_eyes_single_d.gnfp` (olhos).
- UVs lidas de `0x1C/0x1E` no vértice (half floats); a faixa validada é `u=[0.433, 1.0]`, `v=[0.0, 0.499]`. O offset anterior (`0x18/0x1A`) pertencia a outro atributo e causava UVs em `[-1, 1]`.

4.5. Integração da textura no Vulkan

- `UploadTexture(const GnfTexture&)` cria `VkImage` com o formato vindo do GNF, staging buffer, `vkCmdCopyBufferToImage` e transição `UNDEFINED → TRANSFER_DST → SHADER_READ_ONLY_OPTIMAL`.
- A imagem é criada com `mipLevels = 12`; cada mip é copiado com seu próprio `bufferOffset` e `imageExtent`. O sampler usa `maxLod = 11`.
- `VkSampler` com filtro linear e `VkImageView` 2D cobrindo todos os níveis.
- Descriptor set layout/pool/set com `sampler2D` em `set=0, binding=0`; bind no `BeginFrame`.
- Shader de vértice recebe UV em `location 2` (`R32G32_SFLOAT`) e repassa `vUV`; o fragment shader combina a amostra com a iluminação existente.
- SPIR-V regerado com `glslang` (vert: 1756 bytes; frag: 1896 bytes) e embutido em `shader_bytecode.h`.

> **Log de validação:** `textureCompressionBC: suportado`, `UVs da malha: u=[0.432617, 1.0] v=[0.0, 0.499023]` e `Textura enviada para GPU: astro_lowpoly_costume_d.gnfp 2048x2048 formato=146 bytes=5592432`.

5. Formato GNF — descobertas e implementação

Os arquivos `.gnfp` do jogo usam contêiner GNF versão 4 ou 5. O cabeçalho começa com o magic `"GNF "` e o campo `contentsSize` define onde os dados de textura começam.

5.1. Layout do contêiner

Offset	Campo	Observação
`0x00`	`magic[4]`	`"GNF "` (0x20464E47 em little-endian).
`0x04`	`contentsSize: u32`	v4: 0x0FF8 → dados em 0x1000; v5: 0xFFF8 → dados em 0x10000.
`0x08`	`version: u8`	4 ou 5 nos arquivos analisados.
`0x09`	`numTextures: u8`	Normalmente 1.
`0x0A`	`alignment: u8`	Exponente de alinhamento.
`0x0B`	`reserved: u8`	—
`0x10`	Descritor de textura (v4)	8 dwords (0x20 bytes).
`0x28`	Descritor de textura (v5)	8 dwords (0x20 bytes).
`8 + contentsSize`	Início dos dados	Mip 0 começa aqui, em blocos comprimidos tileados.

5.2. Descritor de textura (8 dwords)

Dword	Conteúdo identificado	Como interpretar
0	Base address / offset	Zero nos arquivos GNF.
1	Format ID + alinhamento	`formatId = (reg1 >> 16) & 0xFFFF` ; `alignExp = reg1 & 0xFF` .
2	Dimensões	`widthBlocks = (reg2 & 0x3FFF) + 1` ; `height = ((reg2 >> 14) & 0x3FFF) + 1` .
3	Mips, tile e tipo	`baseLevel=(reg3>>12)&0xF` ; `lastLevel=(reg3>>16)&0xF` ; `tile=(reg3>>20)&0x1F` ; `type=(reg3>>28)&0xF` .
4	Depth / pitch	Campos ainda não usados no carregador atual.
5	Base/last array	Relevante para arrays/cubemaps (não tratado ainda).
6	Flags de compressão / LOD	Não usado nesta fase.
7	Tamanho do payload	Tamanho dos dados de textura em bytes.

> **Dimensões para formatos BC:** a largura é armazenada em *blocos de 4 pixels* e a altura em pixels. Portanto: `width = widthBlocks * 4` e `height = heightField` .
> Exemplo: campo 512 x 2048 → textura real 2048x2048.

5.3. Formatos observados

Format ID	Formato	VkFormat	Bytes/bloco	Uso típico
`0xCB30`, `0xCB60`	BC7 sRGB	146 (`BC7_SRGB_BLOCK`)	16	Diffuse (`_d`)
`0xCB10`	BC7 UNORM	145 (`BC7_UNORM_BLOCK`)	16	Normal (`_n`)
`0xCB50`	BC7 UNORM	145	16	Mask
`0xCAF0`, `0x8AF0`	BC4 UNORM	139 (`BC4_UNORM_BLOCK`)	8	Roughness / AO (`_r`, `_ao`)

Casos raros como `0x8B60` (decals) ainda usam o fallback BC7 UNORM. `0x8AF0` é tratado como BC4 no código atual.

5.4. Tile mode e detile

Os tile modes observados foram **9 em v5** e **5 em v4**. Ambos se comportam como **standard 64 KB** nos testes visuais. O carregador atual trata qualquer `tileMode != 0` como standard 64 KB.

O detile usa as equações GFX10 de 64 KB. Para blocos de 16 bytes (BC7), o bloco cobre 64x64 blocos (256x256 pixels). Para 8 bytes (BC4), cobre 128x64 blocos (512x256 pixels).

> **Investigação de referência (AnyPS5/FreeGNM):** as equações `0x109` do AnyPS5 são **thick (3D)** e não se aplicam ao padrão thin 64 KB; a comparação direta feita anteriormente era inválida. O `kLog2BlockThin64KB` confirma os extents usados localmente (4 B: 128×128; 8 B: 128×64; 16 B: 64×64). O conversor `gnf-conv` do FreeGNM foi compilado, mas crasha ao ler GNF v5 (access violation), portanto não serve como validação direta. A fórmula local permanece a validada pelos previews.

```
// 16 bytes por elemento (BC7)
off = ((x<<6)&0x00C0) ^ ((x<<7)&0x0200) ^ ((x<<8)&0x0800) ^
      ((x<<9)&0x2000) ^ ((x<<10)&0x8000) ^
      ((y<<4)&0x0030) ^ ((y<<6)&0x0100) ^ ((y<<7)&0x0400) ^
      ((y<<8)&0x1000) ^ ((y<<9)&0x4000);

// 8 bytes por elemento (BC4)
off = ((x<<3)&0x0008) ^ ((x<<5)&0x00C0) ^ ((x<<6)&0x0200) ^
      ((x<<7)&0x0800) ^ ((x<<8)&0x2000) ^ ((x<<9)&0x8000) ^
      ((y<<4)&0x0030) ^ ((y<<6)&0x0100) ^ ((y<<7)&0x0400) ^
      ((y<<8)&0x1000) ^ ((y<<9)&0x4000);
```

5.5. Dados carregados atualmente

`GnfTexture.data` contém **todos os mips já destilados**, concatenados em ordem linear de blocos, prontos para upload direto em uma imagem Vulkan BC7/BC4. Cada nível é descrito por `GnfMipLevel` (largura, altura, offset e tamanho). Para a textura de corpo, os 12 níveis totalizam **5.592.432 bytes**, abaixo do payload de 5.636.096 bytes, o que confirma o empacotamento sequencial dos mips no GNF v5.

6. Validações realizadas

6.1. Build e execução

Item	Resultado
Compilação	Sucesso com Clang/LLVM-MinGW UCRT.
Execução	Processo inicia, cria viewport e permanece estável.
Swapchain	714x438, 3 imagens, formato 44.
Pipeline	Criado com sucesso.
Malha	1236 vértices, 5598 índices, 8 draw ranges JXMD (5598 índices cobertos).
Materiais/submeshes	`GEOM` fornece `indexOffset/indexCount`; `MESH` fornece o material (0–3). Filtro de debug: `ASTRO_DEBUG_MATERIAL`.
Primeiro frame	"Primeiro frame apresentado no viewport." registrado em log.
Compressão BC	`textureCompressionBC: suportado`; feature habilitada no dispositivo.
Upload de textura	`astro_lowpoly_costume_d.gnfp`, 2048×2048, formato 146 (BC7 sRGB), 12 mips, 5.592.432 bytes.
Sampler	`CLAMP_TO_EDGE` em U/V/W; filtro linear, mip linear e `maxLod=11`.
Shaders	UV + `sampler2D`; SPIR-V vert 1756 B / frag 1896 B.
UVs da malha	`u=[0.4326, 1.0]`, `v=[0.0, 0.4990]`; half-floats em `0x1C/0x1E`.
Captura de frame	BMP 714×438 gerado via `vkCmdCopyImageToBuffer`; modelo e textura visíveis no frame.

6.2. Testes do parser GNF

Arquivo	Dimensões	Mips no arquivo	Formato	Tile	Mip 0
`arbot_eyes_single_red_d.gnfp`	2048x2048	12	BC7	9	4096 KB
`bnd_tex_rock_d.gnfp`	512x512	10	BC7 sRGB	5	256 KB
`aerial_aquarium_fish_grate_ao.gnfp`	1024x1024	11	BC4	9	512 KB

Além dos metadados, foram gerados previews temporários com decodificação BC7 e detile 64 KB. O resultado visual do `bnd_tex_rock_d` apresenta a textura de rocha coerente; os demais previews também exibem conteúdo coerente, confirmando dimensões, formato e layout de tile.

6.3. Investigação do detile e do suporte BC7

- **AnyPS5:** as equações do modo `0x109` são thick/3D; `EquationSwizzleMode(kStandard64KB)` retorna 0. Não existe equação thin standard 64 KB nessa referência, então a comparação anterior não era aplicável.
- **FreeGNM:** clonado e compilado como referência; o `gnf-conv` crasha com GNF v5 (exit `-1073741819`, access violation), sem gerar PNG. Não foi possível usá-lo para validar o arquivo analisado.
- **Vulkan:** identificado que `VkDeviceCreateInfo` não passava `pEnabledFeatures`. Corrigido com `VkPhysicalDeviceFeatures` completo, carregamento de `vkGetPhysicalDeviceFeatures` e habilitação de

`textureCompressionBC`.

- **Layout de `VkBufferImageCopy`:** o campo `imageOffset` usava `VkOffset2D`; o driver interpretava `imageExtent` deslocado e copiava apenas uma linha (2048×1) da textura. Corrigido para `VkOffset3D`, permitindo o upload completo dos 12 mips e a captura do frame inteiro.
- **Cadeia de mips:** os tamanhos lineares dos 12 níveis somam 5.592.432 bytes e o payload tem 5.636.096 bytes; os níveis são sequenciais e o detile de cada mip foi validado pela captura com mip chain ativa.
- **Submeshes/materiais JXMD:** o chunk `GEOM` fornece `indexOffset` e `indexCount` por draw; o chunk `MESH` fornece o índice de material (0–3). A soma dos 8 ranges é exatamente 5.598 índices. O filtro `ASTRO_DEBUG_MATERIAL` isola um material para inspeção visual.

> **Validação visual determinística:** `ASTRO_CAPTURE_FRAME=1` gera `astro_frame_capture.bmp` lendo a imagem da swapchain na GPU. O frame contém o modelo do Astro com a textura difusa do corpo (`astro_lowpoly_costume_d`) aplicada, confirmando upload, descritor, sampler e UVs.

7. Limitações atuais

- O corpo já usa a textura difusa correta, os UVs estão em `[0,1]` e os 8 ranges/4 materiais JXMD são desenhados separadamente; porém o mapeamento material→textura ainda não foi resolvido, então a textura de olhos está carregada no cache mas não é aplicada às partes correspondentes.
- O mip 0 e a cadeia completa são carregados, porém a validação de cada nível é apenas por contagem/tamanho e captura; ainda não há comparação pixel a pixel com um decodificador de referência.
- Não há escolha de sampler por material (wrap, filtro, anisotropia) nem suporte a múltiplos mapas (normal, roughness, AO, mask); o sampler atual é global em `CLAMP_TO_EDGE`.
- Arrays, cubemaps e casos raros de format ID (`0x8B60`, decals) não foram validados a fundo.
- A malha JXMD real ainda não substituiu totalmente o fallback procedural.
- O tile mode `5` / `9` é tratado como standard 64 KB; o mapeamento exato para o enum GnmTileMode ainda não foi confirmado, embora os previews e a captura sustentem a fórmula atual.

8. Próximos passos recomendados

1. **Mapear materiais para texturas:** os ranges de índice e o material (0–3) já são extraídos do `GEOM` / `MESH`; falta associar cada material às texturas `astro_lowpoly_costume_d`, `astro_lowpoly_costume_e` e `lowpoly_eyes_single_d` (e futuramente normal/roughness).
2. **Refinar sampler por material:** o atlas atual usa `CLAMP_TO_EDGE`; adicionar anisotropia e permitir `REPEAT` apenas nos materiais que exigem tile.
3. **Suportar múltiplos mapas do material:** diffuse (BC7 sRGB), normal (BC7 UNORM), roughness/AO (BC4), com descritores e combinação no fragment shader.
4. **Consolidar a malha JXMD real** e validar o conjunto de atributos (posição, normal, UV, tangente) contra os shaders.
5. **Adicionar validação de camadas Vulkan** para checar descritores, layouts e sincronização em builds de debug.
6. **Evoluir para arrays/cubemaps** e mapear os format IDs restantes (`0x8B60` e variantes).

> **Marco atual:** textura difusa BC7 com 12 mips carregada, UVs validados em `[0,1]`, sampler `CLAMP_TO_EDGE`, 8 draw ranges/4 materiais JXMD e captura confirmando o corpo branco/azul com a textura correta. O próximo foco é o mapeamento material→textura e a validação com camadas Vulkan.

9. Arquivos relevantes

Caminho	Papel
`src/assets/asset_loader.cpp`	Parser GNF v4/v5, classificação de formato e detile do mip 0.
`src/assets/asset_loader.h`	Estruturas `GnfHeader`, `GnfSurfaceInfo` e `GnfTexture`.
`src/gpu/vulkan_backend.cpp`	Swapchain, render pass, pipeline, buffers, draw e logs Vulkan.
`src/gpu/vulkan_backend.h`	API do backend e membros de estado Vulkan.
`src/gpu/vulkan_types.h`	Subconjunto de tipos Vulkan usado pelo projeto (formatos BC4/BC7).
`src/gpu/vulkan_loader.h / .cpp`	Carregamento dinâmico de `vulkan-1.dll`, incluindo `vkGetPhysicalDeviceFeatures`.
`src/gpu/shader_bytecode.h`	SPIR-V embutido de `astro.vert` e `astro.frag` (UV + sampler).
`src/gpu/shaders/astro.vert`	Shader de vértice com posição/normal e UV (`location 2`).
`src/gpu/shaders/astro.frag`	Shader de fragmento com amostragem `sampler2D` + iluminação.
`src/assets/jxmd_parser.cpp/.h`	Parser de malha JXMD e estrutura `JxmdVertex` (inclui UV).
`src/main.cpp`	Janela, viewport filho, integração de UI e chamadas ao backend.
`build_clang.bat`	Build oficial do executável.

10. Comandos e logs

```
Build:
  cd "C:\Users\italo\Desktop\recombilação jogo ps5"
  cmd /c build_clang.bat

Execução:
  build\clang\AstroBot_PC.exe

Logs:
  astro_launch.log
  vulkan_backend.log

Dados do jogo:
  D:\Games\abaixar\jogo\PPSA21567\data

Validação esperada no log Vulkan:
  [VulkanBackend] textureCompressionBC: suportado
  [VulkanBackend] Textura enviada para GPU: ... formato=146

Captura determinística de frame (Windows PowerShell):
  $env:ASTRO_CAPTURE_FRAME='1'
  build\clang\AstroBot_PC.exe
  # gera astro_frame_capture.bmp no diretório de trabalho
```

11. Observações para a próxima sessão

- O parser GNF já está estável para v4/v5 e para os formatos BC7/BC4 observados.
- O detile de 64 KB foi validado visualmente; não voltar a tratar os dados como RGBA linear.
- As equações `0x109` do AnyPS5 são thick/3D; não usar como referência para thin standard 64 KB.
- Dimensões nunca devem ser inferidas pelo tamanho do arquivo: usar o descritor.
- Compressão BC precisa estar habilitada via `pEnabledFeatures`; sem isso, imagens BC7 podem ser inválidas.
- `VkBufferImageCopy.imageOffset` é `VkOffset3D` (12 bytes); não reduzir para `VkOffset2D`, pois isso desalinha `imageExtent` e corrompe uploads/capturas.
- O `gnf-conv` do FreeGNM não é referência para GNF v5 (crasha); usar os previews locais e o parser atual.
- Usar `ASTRO_CAPTURE_FRAME=1` para validar o viewport sem depender de `PrintWindow`.
- Manter o fallback de formato BC7 UNORM para IDs desconhecidos até mapear todos os casos.
- O próximo marco verificável é: mapeamento material→textura (corpo/olhos) e validação com camadas Vulkan; ranges/submeshes já são extraídos e o sampler do atlas está em `CLAMP_TO_EDGE`.
